

Modül 10: İstisnalar

Java istisnaları iki aileye ayırır ve birini derleyiciye denetletir. Bunu yapan neredeyse tek dildir, ve bunun iyi bir fikir olup olmadığı tartışması yirmi beş yıldır sürüyor.

Bu modül mekanizmayı anlatır, tartışmayı adil biçimde aktarır ve insanlara gerçek hata ayıklama süresi kaybettiren üç başarısızlık biçimini gösterir: yutulan bir istisna, kaybolan bir sebep ve asıl sorunu gizleyen bir `close()` çağrısı.

Neler Öğreneceksiniz

- Denetlenen ve denetlenmeyen ayrımı, ve hangisini fırlatacağınıza nasıl karar vereceğiniz
- `try / catch / finally`, çoklu yakalama ve yakalama sırası
- `try-with-resources`, ters kapatma sırası ve bastırılan istisnalar
- `finally` içindeki `return` neden her stil kılavuzunda yasak
- Özel istisnalar ve insanların unuttuğu kurucu argümanı

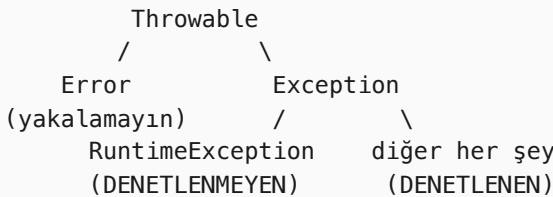
Başka Bir Dilden Geliyorsanız

	Python / JS	Java
Derleyicinin zorunlu kıldığı ele alma	yok	denetlenen istisnalar
Birkaç tipi yakalama	<code>except (A, B)</code>	<code>catch (A B e)</code>
Temizlik	<code>finally</code> , <code>with</code>	<code>finally</code> , <code>try-with-resources</code>
Kaynak protokolü	<code>__enter__ / __exit__</code>	<code>AutoCloseable</code>
İstisna zincirleme	<code>raise X from e</code>	<code>new X(msg, cause)</code>

Gerçekten yeni olan tek fikir denetlenen istisnalar. Diğer her şeyi zaten başka bir adla biliyorsunuz.

Ders

İki aile



- **Denetlenmeyen** (`RuntimeException` ve altı): derleyici bir şey söylemez. Bunlar programlama hatası demektir. Çözüm kodu değiştirmektir, onu ele almak değil.
- **Denetlenen** (`Exception` altındaki diğer her şey): derleyici sizi yakalamaya ya da bildirmeye zorlar.
- **Error**: JVM başı dertte. `OutOfMemoryError`, `StackOverflowError`. Bunları yakalamayın, içeriden düzeltemezsiniz.

Bilinmeye değer bir ayrıntı, `CheckedVsUnchecked.java` dosyasından:

```
unchecked: NumberFormatException -> For input string: "12x"  
unchecked: NumberFormatException -> Cannot parse null string
```

`Integer.parseInt(null)` çağırısı `NullPointerException` **değil**, `NumberFormatException` fırlatır. Javadoc okumak yerine tipi tahmin etmek, hiç tetiklenmeyen bir `catch` bloğu yazmanın yoludur.

Çoklu yakalama iki tipi aynı biçimde ele alır, ve değişken örtük olarak final'dır. **Yakalama sırası önemlidir**: üst tip alt tiplerinden sonra gelmeli, ve derleyici erişilemez bir catch'i orada bırakmak yerine reddeder.

Tartışma, adil biçimde

Lehine: çağırının kurtulabileceği bir hata, tip sisteminde görünür olmalıdır. Bir dosya var olmayabilir. Bu bir hata değil, hayatın olağan akışı, ve derleyicinin size bunu düşündürmesi bir özellik.

Aleyhine: pratikte insanları derleyiciyi susturmak için `catch (Exception e) {}` yazmaya itiyorlar, ki bu hiç denetim olmamasından kesinlikle daha kötü. Soyutlamalardan sızıyorlar, çünkü bir metoda `throws` eklemek yukarı doğru her çağırının imzasını değiştiriyor. Ve lambda'larla kötü uyuyorlar, stream API'sinde fırlatan bir fonksiyon kabul eden hiçbir şey olmamasının sebebi bu.

Java'dan sonra tasarlanan hiçbir dil onları kopyalamadı. Bu kanıttır, ispat değil.

Bugün ne yapmalı:

- Programlama hataları için denetlenmeyen: `IllegalArgumentException`, `IllegalStateException`.
- Denetlenen yalnızca çağırın gerçekten loglayıp pes etmekten başka bir şey yapabildiğinde.
- Asla `throws Exception` bildirmeyin. Çağırana hiçbir şey söylemez ve her şeyi yakalamaya zorlar.

try-with-resources

Mutlu yol dikkate değer değil. Tasarımın kendini gösterdiği yer hata yolu. `TryWithResources.java` dosyasından:

```
open A
open B
body running
close B (and failing)
close A (and failing)
caught: body failed
  suppressed: close B failed
  suppressed: close A failed
```

O çıktıdan iki şey çıkar:

1. **Kaynaklar ters sırada kapanır.** B en son açıldı, ilk kapandı. Bir kaynak diğerini sardığında bu önemlidir.
2. **Kapatma hataları yerine geçmez, bastırılır.** Yakaladığınız istisna gövdeninkidir; `close()` hataları ona iliştilir ve `getSuppressed()` ile okunabilir.

İkisi de elle yazılan eşdeğerinin bunları yanlış yapmasından dolayı var:

```
Resource r = null;
try {
    r = open();
    use(r);
} finally {
    if (r != null) r.close(); // if this throws, it REPLACES the real exception
}
```

Bir `close()` hatası asıl sorunu gizler ve gerçek sebep kaybolurdu.

Gövde erken döndüğünde de `close()` çalışır, dolayısıyla bunun için asla bir `finally` gerekmez.

finally ve istisnaları nasıl yediği

`finally` her zaman çalışır. Bu hem söz hem tuzaktır. `FinallySwallows.java` dosyasından:

```
static int swallowsTheException() {
    try {
        throw new RuntimeException("you will never see this");
    } finally {
        return 42;
    }
}
```

Çıktı: `42`. İstisna yok oldu. Log yok, yığın izi yok, hiçbir şey yok.

`finally` içindeki bir `return`, `try` bloğunun yaptığı her şeyi atar, uçuştaki bir istisna dahil. Aynı mekanizma iyi bir dönüş değerini de sessizce ezer, ki incelemede fark edilmesi çok daha zordur.

`finally` bloğuna asla `return`, `break` ya da `continue` koymayın. Her stil kılavuzu bunu yasaklar ve sebebi budur.

Ayrıca JVM çıkarsa `finally` çalışmaz. `try` içindeki `System.exit(0)` onu tamamen atlar. "Her zaman çalışır", normal bir metod çıkışı içinde demektir.

Özel istisnalar ve sebep

`CustomExceptions.java` dosyasından, aynı sarmalama iki kez yapılmış:

```
--- with the cause preserved ---
  caused by: java.io.IOException: users/7.json: no such file
  root cause: java.io.IOException: users/7.json: no such file

--- with the cause thrown away ---
  caused by: null
  the original IOException is gone. good luck.
```

Sebep ikinci kurucu argümanıdır ve onu unutmak istisna işlemede en yaygın hatadır. Onsuz yığın izi yeniden fırlattığınız yerden başlar ve gerçekte neyin başarısız olduğu hakkında hiçbir şey söylemez.

Özel bir istisnaya her zaman `Throwable` sebep alan bir kurucu verin.

Mesaja gömmek yerine yapılandırılmış veri taşıyın, böylece çağırانlar kimliği bir metinden geri ayrıştırmak yerine sorabilsin.

Denetlenen mi denetlenmeyen mi? `extends Exception` denetlenendir, `extends RuntimeException` değildir. Çağırانın loglayıp pes etmekten başka bir şey gerçekten yapıp yapamayacağını sorun.

Çalıştırın

```
java modules/10-exceptions/examples/CheckedVsUnchecked.java
java modules/10-exceptions/examples/TryWithResources.java
java modules/10-exceptions/examples/FinallySwallows.java
java modules/10-exceptions/examples/CustomExceptions.java

# Bilerek başarısız olur.
java modules/10-exceptions/examples/UnhandledChecked.java

./scripts/verify-examples.sh modules/10-exceptions
```

Sık Yapılan Hatalar

Boş catch bloğu.

```
try { readConfig(path); } catch (IOException e) { }
```

Derlenir, derleyiciyi susturur, ve görünür bir hatayı sessizce yanlış iş yapan bir programa çevirir. Java'da yazabileceğiniz en kötü satır budur, ve denetlenen istisnaların bunu üretmesi özelliğe karşı en güçlü argümandır.

Sebebi kaybetmek. Catch bloğunun içinde `throw new MyException("failed")` yazmak, hata ayıklamak için ihtiyacınız olan her şeyi atar.

`finally` bloğunda `return`. Yukarıda anlatıldı. İstisnaları yer.

Her şeyi kapsamak için `Exception` yakalamak. Beklenen hataların yanında hataları da yakaladınız, nasıl ele alacağınızı bilmedikleriniz dahil.

`Throwable` ya da `Error` yakalamak. `OutOfMemoryError` durumundan kurtulamazsınız, ve onu yakalamak bir çökmeyi bir asılı kalmaya çevirir.

Akış kontrolü için istisna kullanmak. Döngüden çıkmak için fırlatmak yavaştır, çünkü yığın izi oluşturmak pahalıdır, ve mantığı okuyandan gizler.

`throws Exception` bildirmek. Çağıranlara her şeyin artık onların sorunu olduğu dışında hiçbir şey söylemez.

Özet Çıkarımlar

- **Denetlenmeyen hata demek, denetlenen öngörülebilir başarısızlık demektir.** Çağıranın gerçekte ne yapabileceğini sorarak seçin.
- **Asla boş catch bloğu yazmayın.** Gerçekten ele alamıyorsanız denetlenmeyen bir istisnaya sarın ve yolculuğuna devam etsin.
- **try-with-resources ters sırada kapatır ve yerine geçmek yerine bastırır**, ki elle yazılan temizlik bunu yıllarca yanlış yaptı.
- `finally` içindeki `return` istisnaları sessizce atar.
- **Sarmalarken her zaman sebebi geçirin**, ve özel istisnalara her zaman sebep kurucusu verin.
- **Asla `throws Exception` bildirmeyin.**

Ödev

[homework/README.md](#)

Küçük bir yapılandırma yükleyici kurun, sonra üç istisna yanlış kullanımını yeniden üretip her birini düzeltin. Referans çözüm [solutions/10-exceptions/](#) altında.